

Aula 1 — Memória e a folha de dados

Uma conta que não fecha, e o registrador que ela obriga a existir

Microcontroladores — DENE/UFMT

OBJETIVOS

- Localizar na folha de dados o endereço de um registrador de função especial, e explicar por que LATD é um nome e não uma variável.
- Distinguir os três espaços de memória do PIC18F4550 e justificar por que uma constante e uma variável não moram no mesmo lugar.
- Derivar a existência do BSR a partir da largura do campo de endereço na codificação da instrução.
- Descrever o access bank como um mapa de 256 endereços que evita a troca de banco, e prever quando o compilador é obrigado a emitir MOVLB.
- Especificar o espaço de dados com precisão suficiente para implementá-lo em Python, incluindo o comportamento da região não implementada.

1 A pergunta que sobrou do encontro 0

No primeiro programa escrevemos `LATDbits.LATD0 = 1` e afirmamos, sem provar, que aquilo era uma escrita em memória. Hoje a afirmação vira endereço.

Abra a folha de dados do PIC18F4550 na tabela de registradores de função especial e procure LATD. Ele está em **0xF8C**.

LATD não é uma variável. Nenhuma memória foi reservada para ele, nenhum `malloc` aconteceu, e o compilador não escolheu onde colocá-lo. O endereço `0xF8C` existia antes do seu programa, existe quando não há programa nenhum gravado, e continuaria existindo se você programasse em assembly ou em Forth.

CONCEITO

O `xc.h` não implementa LATD. Ele apenas informa ao compilador um endereço que está impresso na folha de dados.

1.1 A parede do encontro 0, agora legível

O encontro 0 mostrou esta linha e pediu que vocês não tentassem entendê-la ainda:

```
#define LATD (*(volatile unsigned char *) 0xF8C)
```

Agora ela se lê da direita para a esquerda, e cada pedaço já foi apresentado.

`0xF8C` — o endereço. Está na tabela de registradores da folha de dados; a §3 vai mostrar por que ele tem exatamente três dígitos hexadecimais, e o que isso obriga a existir dentro do processador.

CONCEITO

`(unsigned char *)` — trate esse número como **ponteiro para um byte**. O compilador não sabia disso: para ele, `0xF8C` era só um inteiro.

`volatile` — o valor pode mudar sem que o programa mude. Sem essa palavra, o compilador tem o direito de ler uma vez e reutilizar o resultado, e é isso que ele faria com um endereço que ele acha que ninguém mais toca.

`*(...)` — a desreferência. Escrever aqui é escrever **naquele** byte.

O resultado tem um nome e é isto o que o `xc.h` faz, algumas milhares de vezes. Nada de mágico aconteceu: alguém digitou os endereços da folha de dados.

E note o que a linha **não** faz: ela não reserva memória, não inicializa nada e não gera uma única instrução. Ela é uma tradução de nome para número, resolvida antes de o compilador começar.

NOTA

O byte em 0x0F8C não foi criado por ela. Ele já estava lá, com transistores presos nele, desde antes de a placa ser ligada.

Exercício 1.0. Escreva a mesma linha para TRISD, que está em 0x0F95, e diga o que aconteceria se você esquecesse `volatile` num programa que lê `PORTB` num laço esperando um botão.

TAREFA

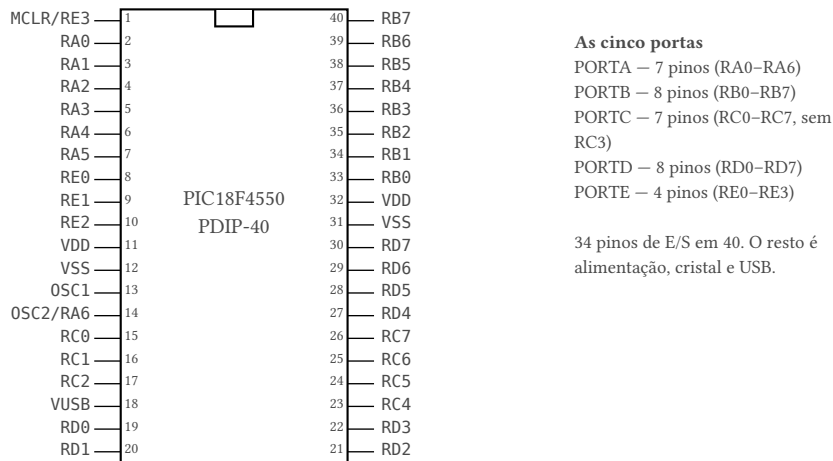


Figura 1: O objeto do semestre. Dos quarenta pinos, trinta e quatro são de entrada e saída; o resto é alimentação, cristal e USB. `LATD0` é o pino 19 — escrever no endereço 0xF8C move elétrons ali.

A partir daqui, a folha de dados é a autoridade do semestre. Quando ela e o material discordarem, ela ganha; quando ela e a placa discordarem, a placa ganha.

2 Três memórias, não uma

Espaço	Tamanho	Endereços	Como se chega lá
Programa (Flash)	32 kB	0x0000–0x7FFF	O contador de programa. É o único que o processador busca instruções
Dados (RAM + SFR)	4096 endereços, 2048 implementados como RAM	0x000–0xFFFF	O campo de endereço das instruções
EEPROM	256 bytes	0x00–0xFF	Não é endereçável. Só através de <code>EEADR</code> , <code>EEDATA</code> e <code>EECON1</code>

Isso é uma arquitetura Harvard: instruções e dados em memórias fisicamente separadas, com barramentos separados. Duas consequências imediatas.

A primeira é que o processador pode buscar a próxima instrução enquanto executa a atual, porque as duas buscas não disputam o mesmo barramento. É disso que sai o desempenho de uma instrução por ciclo, e é assunto do encontro 2.

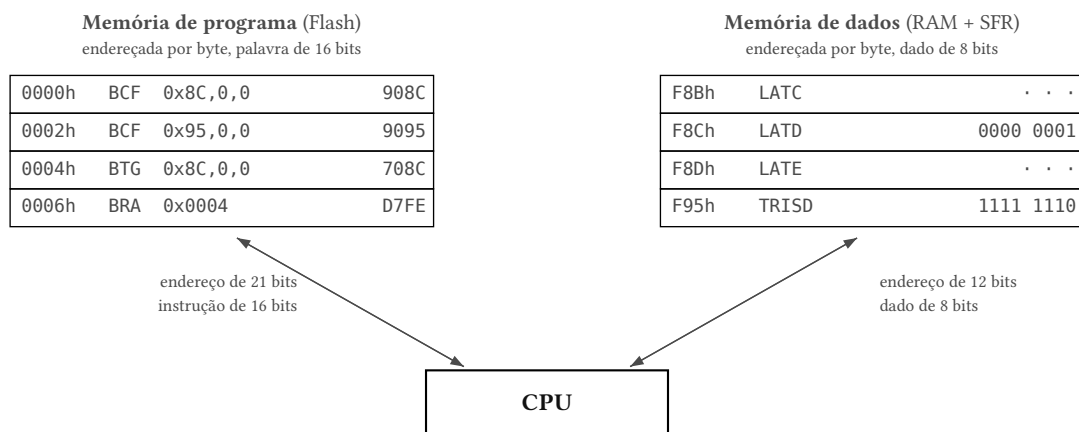


Figura 2: Dois espaços, dois barramentos, duas larguras de endereço. O mesmo número 0x0002 significa coisas diferentes em cada lado, e nenhuma instrução alcança os dois.

A segunda aparece no seu código. Num computador, uma tabela const e um vetor comum moram no mesmo espaço, e um ponteiro serve para os dois. Aqui a tabela está na Flash e o vetor na RAM, e chegar a cada um exige uma instrução diferente. Uma tabela de constantes grande demais para a RAM é lida com TBLRD, que é outro mecanismo — voltaremos a ele quando a tabela existir.

A EEPROM é o caso mais claro de “memória que não é endereço”. Você não escreve nela; você escreve o endereço em EEADR, o dado em EEDATA, e então manda EECON1 executar. É um periférico com cara de memória, e o contraste com a EEPROM externa por I²C é a espinha do encontro 11.

NOTA

3 O espaço de dados

3.1 Doze bits

Faixa	Bancos	O que há
0x000–0x7FF	0 a 7	2048 bytes de RAM de uso geral. No 4550, os bancos 4 a 7 podem ser tomados pelo módulo USB
0x800–0xEFF	8 a 14	Não implementado. Leitura devolve 0, escrita é descartada
0xF00–0xFFFF	15	Os registradores de função especial. LATD mora aqui

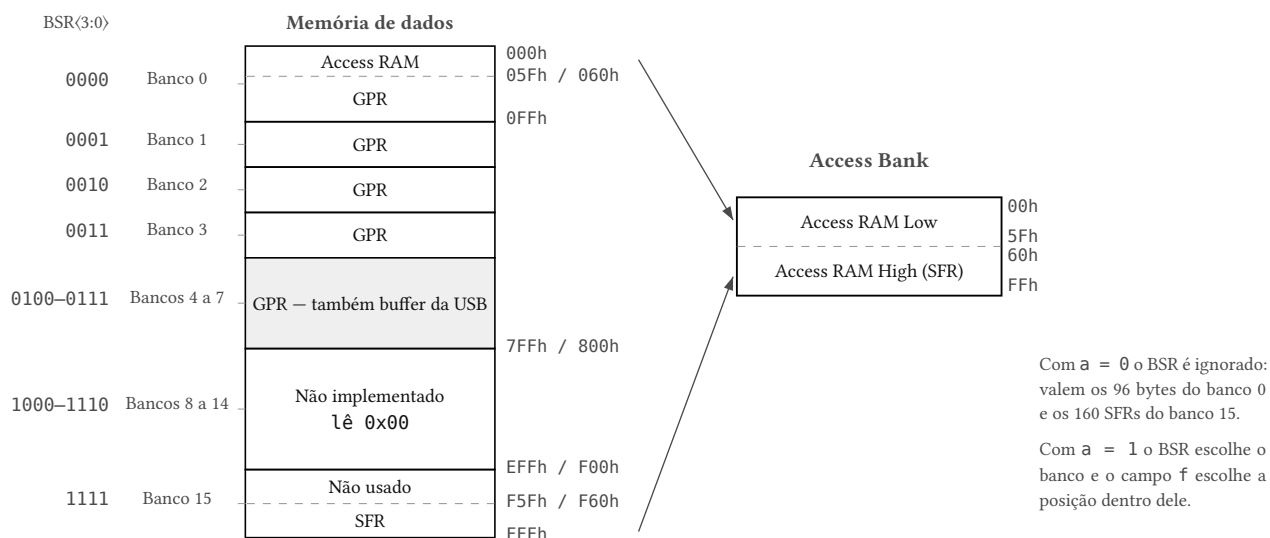


Figura 3: O espaço de dados inteiro. Mais da metade dele não existe fisicamente e lê 0x00 — é essa região que o simulador do R2 precisa reproduzir, e reproduzir o vazio é parte da especificação.

Some tudo: 4096 endereços, que é 2^{12} . Três dígitos hexadecimais. É por isso que o curso usa hexadecimal — não por tradição, mas porque cada dígito hexa é exatamente meio byte de endereço, e o mapa acima só é legível nessa base.

A largura do endereço define o tamanho do espaço, e não o contrário. O PIC18F4550 tem 2048 bytes de RAM, mas **fala** em 12 bits — então existe espaço para 4096, e mais da metade dele é vazio. Esse vazio não é desperdício: é o que permite que um irmão maior da mesma família tenha mais RAM sem mudar uma linha da arquitetura nem uma instrução do conjunto.

CONCEITO

3.2 A conta que não fecha

Agora abra a codificação da instrução MOVWF na folha de dados:

```
MOVWF f, a      0110 111a ffff ffff
```

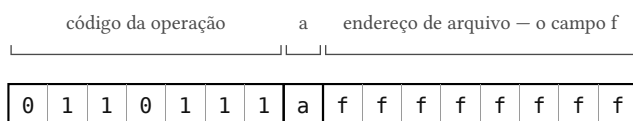


Figura 4: A palavra de instrução do PIC18 tem dezesseis bits, e o endereço do destino ocupa oito deles.

Dezesseis bits. Sete são o código da operação, um é chamado a, e sobram **oito** para o campo f — o endereço do destino.

Antes de virar a página, responda em uma linha cada:

TAREFA

- Quantos endereços distintos oito bits alcançam?
- Quantos endereços o espaço de dados tem?
- Quantos bits estão faltando, e onde eles poderiam estar guardados, já que não cabem na instrução?

3.3 O BSR

Existe um registrador de 4 bits chamado **Bank Select Register**. O endereço efetivo é a concatenação:

endereço de 12 bits $\leftarrow$ BSR[3:0] : f[7:0]

Os quatro bits do BSR escolhem um dos dezesseis **bancos** de 256 bytes; os oito bits da instrução escolhem a posição dentro do banco. Trocar de banco é uma instrução: MOVLB k.

O BSR é modo, não dado. Ele muda o significado de todas as instruções seguintes sem aparecer em nenhuma delas. Uma rotina que altera o BSR e não o restaura quebra o código que vier depois, e a quebra é silenciosa — o programa escreve no endereço certo do banco errado.

CONCEITO

Este é o primeiro exemplo do semestre de estado global implícito. O último será a mesma coisa com interrupções, no encontro 7, e lá o dano é pior porque o momento da quebra é imprevisível.

3.4 O access bank

Trocar de banco toda vez que se toca num SFR seria caro: uma instrução extra antes de cada acesso. O bit a da instrução existe para evitar isso.

Bit a	Endereço efetivo
a = 1	Banco selecionado pelo BSR. Endereço = BSR:f
a = 0	Access bank : o BSR é ignorado e vale um mapa fixo

O mapa fixo divide os 256 valores possíveis de f em duas metades desiguais:

f	Endereço efetivo	O que se alcança
0x00–0x5F	0x000–0x05F	Os 96 primeiros bytes do banco 0 — onde o compilador põe as variáveis mais usadas
0x60–0xFF	0xF60–0xFFFF	Os 160 SFRs mais usados

96 mais 160 é 256. O access bank é uma janela de 256 endereços montada com dois pedaços que não são vizinhos: o começo da RAM e o fim dos SFRs. Com $a = 0$ você alcança essa janela sem tocar no BSR.

LATD está em 0xF8C. Escreva os dois modos de acessá-lo e diga qual custa menos:

TAREFA

(a) via BSR; (b) via access bank.

Nem todo SFR está no access bank. Os que ficam abaixo de 0xF60 — no 4550, os registradores de endpoint da USB — só são alcançáveis com `MOVLB 0xF` antes. Se o seu código toca a USB e depois um LED, o BSR precisa voltar.

ATENÇÃO

Confira a faixa exata na tabela de registradores da folha de dados antes de confiar nesta frase.

4 O que o compilador estava escondendo

```
LATDbits.LATD0 = 1;    -> BSF 0x8C, 0, 0    ; 1 instrução, access bank
minha_variavel = 1;    -> MOVLB 3          ; se a variável caiu no banco 3
                        MOVLW 1
                        MOVWF 0x12, 1
```

O compilador escolhe onde suas variáveis moram, e essa escolha tem preço em instruções. Variável que cai nos 96 bytes de baixo é barata; variável que cai num banco alto custa um `MOVLB` sempre que o banco anterior era outro.

4.1 A parede, fechada

Volte à linha da §1.1, agora com tudo o que esta aula construiu:

```
#define LATD (*(volatile unsigned char *) 0x0F8C)
```

O número tem **doze bits**. O campo de endereço da instrução tem **oito**. A linha pede um acesso que a instrução, sozinha, não sabe codificar — e é essa impossibilidade que obriga o BSR e o access bank a existirem.

CONCEITO

O que o compilador faz com `LATD = 1` depende de onde o endereço cai:

Endereço no #define	O que sai	Por quê
0x0F8C (LATD)	<code>BSF 0x8C, 0, 0</code> — uma instrução	Está no access bank, na metade dos SFRs
0x0050 (RAM baixa)	uma instrução	Está no access bank, na metade dos 96 bytes
0x0300 (banco 3)	<code>MOVLB 3</code> mais a instrução	Fora do access bank: precisa trocar de banco
0x0900	uma instrução, e nada acontece	Região não implementada: escrita descartada

A mesma linha de C, escrita quatro vezes com quatro números, produz quatro comportamentos diferentes — e nenhum deles é escolha sua. Todos são consequência do mapa que esta aula desenhou.

É isso que a §1.1 ainda não podia dizer. Lá a linha era legível **palavra por palavra**; aqui ela é legível **no que custa**.

NOTA

E a última linha da tabela é a mais desconfortável: escrever em 0x0900 compila, executa, não avisa e não faz nada. O compilador não tem como saber que aquele endereço não existe — o mapa está na folha de dados, não na linguagem.

É por isso que o simulador do R2 precisa reproduzir a região não implementada em vez de ignorá-la: reproduzir o vazio é parte da especificação.

No XC8 gratuito, as variáveis locais recebem endereços fixos e estáticos — não existe pilha de dados como você conhece do computador. A pilha do PIC18 tem 31 níveis, é de hardware, guarda só endereços de retorno, e não está no espaço de dados. Ela não serve para variáveis.

NOTA

A consequência prática — funções não reentrantes — só morde quando existe interrupção, e por isso fica para o encontro 7. Registre o fato agora.

5 A memória de programa

Endereço	O que há
0x0000	Endereço de reset. Fixo em hardware
0x0008	Vetor de interrupção de alta prioridade
0x0018	Vetor de interrupção de baixa prioridade
até 0x7FFF	32 kB de Flash, endereçados por byte

A palavra de instrução tem 16 bits e a memória é endereçada por byte: cada instrução ocupa dois endereços, e toda instrução começa em endereço par. O contador de programa anda de dois em dois, e o bit menos significativo dele é fixo em zero — assunto do encontro 2.

No kit, o começo da Flash está ocupado pelo bootloader, e o seu programa começa depois dele. Isso não muda o endereço de reset: quem começa em 0x0000 é o bootloader, e é ele que decide passar o controle ao seu código.

NOTA

Confundir “endereço de reset” com “menor endereço presente no arquivo HEX” já produziu um defeito real no simulador deste curso. A distinção é do encontro 2.

6 O que o R2 tem que implementar

O laboratório da semana que vem é a primeira camada do simulador: o espaço de dados como estrutura em Python, com o esqueleto e os testes já prontos. A especificação é esta aula inteira, resumida em cinco frases:

1. O espaço tem 4096 endereços de um byte.
2. 0x000 a 0x7FF são RAM: escreve e lê de volta.
3. 0x800 a 0xEFF não existem: leitura devolve 0, escrita não tem efeito.
4. 0xF00 a 0xFFF são SFRs: por ora, comportam-se como RAM, com nomes.
5. O endereço efetivo de um acesso depende de *a* e, quando *a* = 1, do BSR.

Um simulador escrito sem cuidado usa uma lista de 4096 posições *e*, com isso, **ganha memória que o chip não tem**. Escreva 0x55 em 0x900, leia de volta, e o Python devolve 0x55; o silício devolve 0.

DIVERGÊNCIA

Esse é o primeiro lugar do semestre onde o simulador mente, e é uma mentira consequente: o compilador nunca aloca ali, então o defeito não aparece em programa gerado por compilador — aparece exatamente quando você escreve o endereço à mão, que é o que vamos fazer no encontro 2. O teste que pega isso é obrigatório no R2.

7 Previsão para o R2

PREVISÃO

P1. Sua classe vai guardar 4096 posições ou 2048? Justifique a escolha pela especificação, não pela economia.

P2. Escrever em 0x900 e ler de volta deve devolver o quê? E escrever em 0xF8C?

P3. Com $BSR = 0x3$, a instrução `MOVWF 0x12, 1` escreve em qual endereço? E `MOVWF 0x12, 0`?

P4. Com $BSR = 0x3$, `MOVWF 0x8C, 0` escreve em qual endereço? A resposta depende do BSR ?

P5. Que teste você escreveria para provar que a sua implementação do access bank não é apenas “somar $0xF00$ quando f é grande”?

Leve esta folha preenchida e o mapa de memória da folha de dados aberto. O esqueleto e os testes serão distribuídos no início da sessão; o tempo é curto para escrever e conferir a especificação ao mesmo tempo.

BANCADA

8 Exercícios

Exercício 1.1. Um colega afirma: “o PIC18F4550 tem 2 kB de RAM, então bastam 11 bits de endereço; os 12 bits são desperdício.”

TAREFA

(a) Ele está certo sobre os 11 bits?

(b) Dê uma razão de arquitetura, não de economia, para o espaço ser de 12 bits.

(c) O que aconteceria com o conjunto de instruções se cada modelo da família usasse a largura mínima para a sua RAM?

Exercício 1.2. Considere o trecho:

TAREFA

```
MOVLB    0x3
MOVWF    0x20, 1
BSF      0x8C, 0, 0
MOVWF    0x20, 1
```

(a) Em que endereço cada `MOVWF` escreve?

(b) O `BSF` alterou o BSR ?

(c) Se o terceiro acesso usasse $a = 1$ em vez de 0, em que endereço ele escreveria — e qual seria o sintoma observável na placa?

Exercício 1.3. O access bank cobre $0x00$ – $0x5F$ da RAM e $0xF60$ – $0xFFFF$ dos SFRs.

TAREFA

(a) Por que a divisão é $96/160$ e não $128/128$?

(b) Uma variável alocada em $0x60$ está no access bank?

(c) Proponha um critério que um compilador poderia usar para decidir quais variáveis colocar abaixo de $0x60$, e diga que informação ele precisaria ter.

Exercício 1.4. Escreva, em Python, o método `le(self, endereco)` que respeita a especificação da §5. Não use `if` em cadeia com mais de três ramos.

TAREFA

No encontro 2: o relógio e o ciclo de busca-execução. A pergunta de abertura é por que um circuito puramente combinacional ainda não é um computador — e a resposta transforma o mapa de memória de hoje em algo que **executa**. Ao final, os pinos do simulador ligam e desligam.

NOTA